

Distributed Search Engine

A progressively-built, MapReduce-powered search engine deployed on AWS EC2 — from single-machine crawling to fully distributed indexing and retrieval.

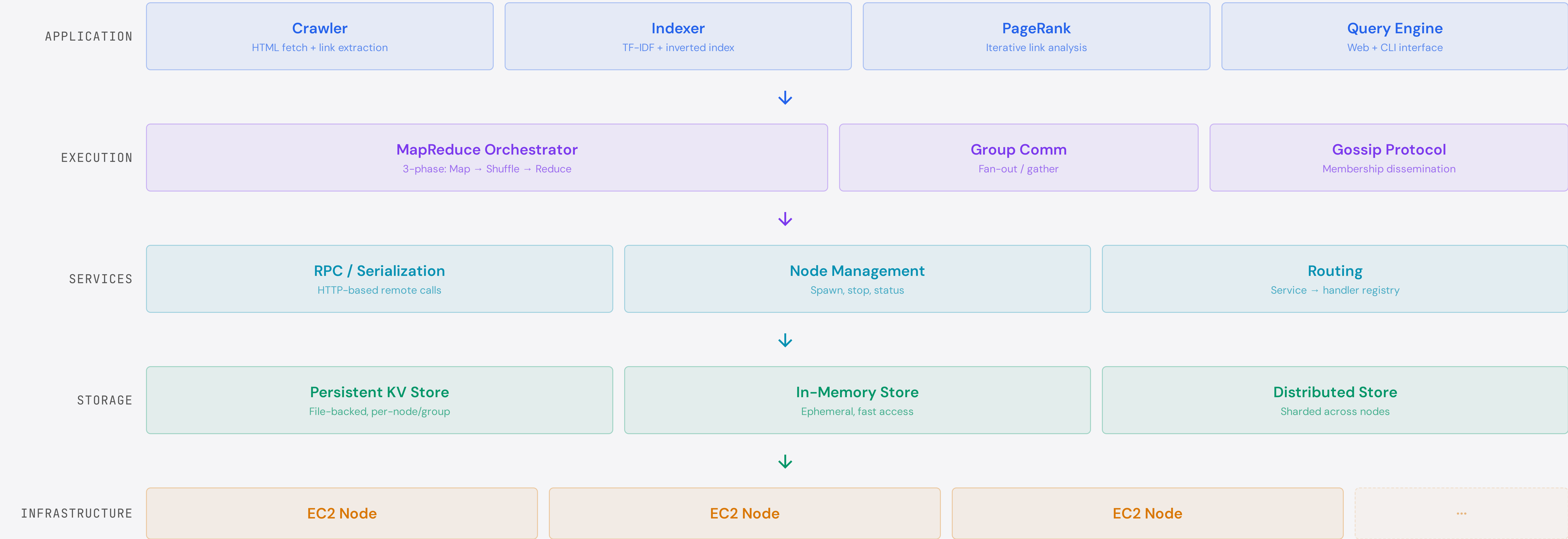
Julia Zdzilowska and Jesus Rodriguez

Brown University
Department of Computer Science
Spring 2026

01 – SYSTEM OVERVIEW

End-to-End Architecture

The system is structured in layers: application-level search components communicate through a group-based distributed services framework, backed by persistent and in-memory storage, all running on AWS EC2 nodes.



02 – PROGRESSIVE BUILD

Milestone Progression

M0	Single-Machine Search Crawler, indexer, query engine — no distribution	279 LoC
M1	Serialization Cross-network object transmission with function support	185 LoC
M2	Communication HTTP-based RPC, node spawning, remote invocation	593 LoC
M3	Groups & Gossip Group membership, gossip protocol, fan-out/gather	250 LoC
M4	Distributed Storage Persistent & in-memory KV stores, sharding by group	1,649 LoC
M5	MapReduce Distributed execution with Map → Shuffle → Reduce pipeline	1,899 LoC
~14K Total Lines of Code		
59 Hours of Development		
5 Milestones Integrated		

03 – EXECUTION MODEL

MapReduce Pipeline

The coordinator registers ephemeral services across all group nodes, serializes mapper/reducer functions, and orchestrates three synchronized phases:



Synchronization: Coordinator tracks completion per phase via `notify` callbacks — advances only when all nodes confirm. Functions are serialized and shipped to remote nodes for local execution.

04 – PERFORMANCE

Key Benchmarks

14,250 Requests/sec (parallel RPC) Local, M2	1,393 Store ops/sec Local, M4
111.6 Docs/sec (MapReduce) Local, M5	44.8ms MapReduce latency/run Local, M5

05 – LOCAL VS. AWS EC2

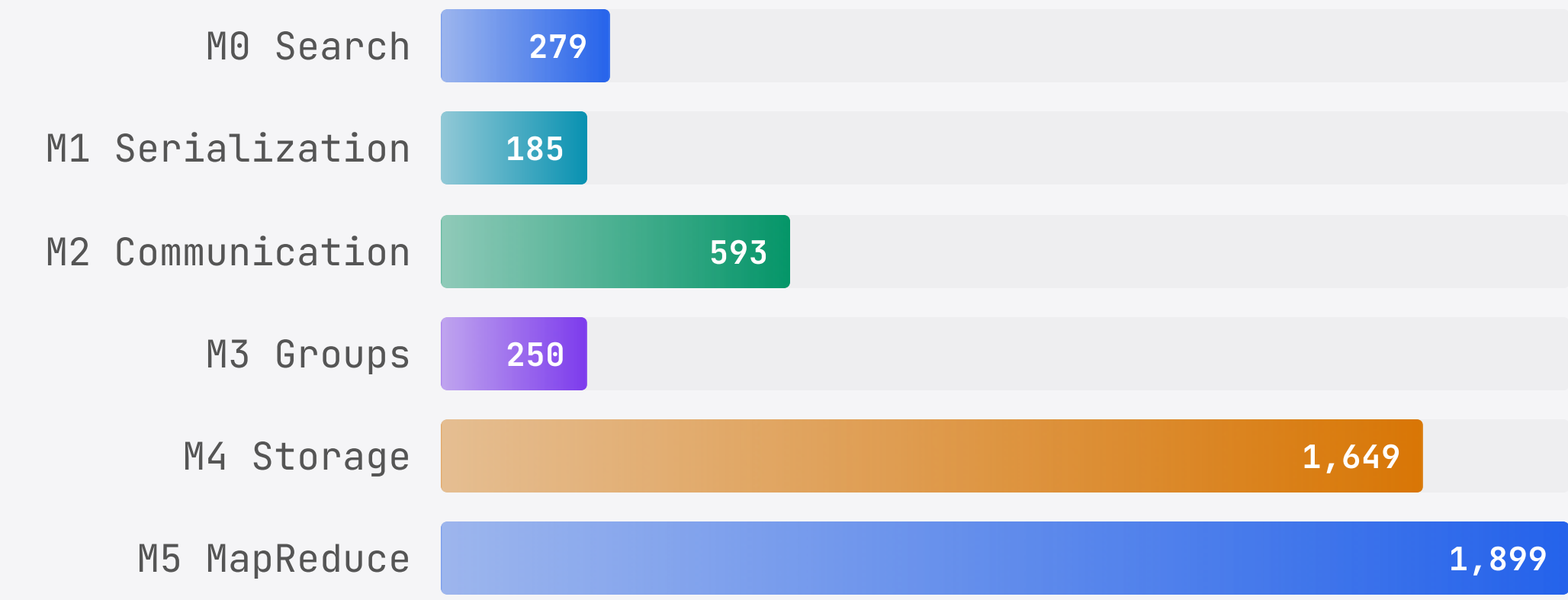
Latency Comparison

OPERATION	LOCAL (APPLE M2)	AWS EC2
Serialization (complex)	44.28 μ s	61.83 μ s
RPC round-trip	~0.07 ms	~0.36 ms
Node spawn	97.99 ms	146.49 ms
KV Store put	2.12 ms	1.29 ms
KV Store get	0.72 ms	0.89 ms

AWS latency is comparable to local for I/O-bound operations (KV store) but shows network overhead for RPC-heavy workloads. Storage ops benefit from EC2's optimized disk I/O.

06 – THROUGHPUT BY MILESTONE

Lines of Code Contributed



07 – COMPONENTS

Module Breakdown

MODULE	ROLE	LOC
local/node.js	HTTP server, request routing	~200
local/store.js	File-backed persistent KV	~160
local/comm.js	RPC client, error handling	~120
all/mr.js	MapReduce orchestrator	~250
all/comm.js	Fan-out/gather broadcast	~150
util/serialization.js	Object/function serializer	185
non-dist/merge.js	Inverted index merger	171

08 – CHALLENGES & SOLUTIONS

Key Technical Decisions

Serializing Functions Across the Wire

MapReduce requires shipping mapper/reducer functions to remote nodes. Built a recursive serializer handling closures, special values (NaN, Infinity), nested objects, and Date/Error types.

MapReduce Phase Synchronization

Coordinator must wait for all nodes to complete each phase before advancing. Implemented a `notify`-based callback barrier with ephemeral service registration/deregistration per job.

Consistent Hash-Based Shuffle

During shuffle, all nodes must agree on data routing. Used deterministic consistent hashing on keys to ensure symmetric placement across the cluster.

Group-Scoped Service Isolation

Every service is scoped to a named group, enabling multiple independent clusters (e.g., crawler group, indexer group) on the same physical nodes.

09 – STACK & ENVIRONMENT

Technology

Node.js	JavaScript ES6+	AWS EC2	HTTP RPC	Jest
natural.js	jsdom	html-to-text	yargs	

Dev environment: Apple M2, 16 GB RAM, 8 cores (local). Ubuntu on Amazon EC2 (deployed). All components tested both locally and on multi-node AWS clusters.

Testing: Comprehensive Jest suite with 4 tiers — unit tests, scenario tests (NCDC, TF-IDF, string matching), extra credit, and instructor-provided integration tests.